LAB 7: Fuzzing for Input Validation Bugs

Bus Transit Fare Processing System — Property-Based Testing with Hypothesis

Student Name: Shashank D	**SRN:** PES1UG24CS433
Course Module: Lab 7 (Fuzz Testing)	**Repository:** L6_L7_10_bus_transit_fare
Testing Framework: Hypothesis 6.170 + Pytest 9.1	**Technique:** Automated Property-Based Fuzzing
Target Script: processor.py	**Test Suite:** test_processor.py

1. Lab Objective & Concept Introduction

The objective of this lab is to explore **fuzz testing** as an automated security, reliability, and quality assurance technique. Traditional unit tests only examine predetermined scenarios anticipated by developers. Fuzzing, particularly **property-based fuzzing** using the **Hypothesis** library, tests what developers *never imagined could go wrong* by automatically synthesizing hundreds of pseudo-random, boundary, and malformed inputs (empty strings, None values, Unicode symbols, special delimiters, negative numbers, and non-numeric characters). The core property under test is **crash resistance**: regardless of malformed inputs, no unhandled exceptions should ever crash the application.

2. Step 1: Environment Setup & Dependency Installation

The testing environment was configured by installing the Hypothesis fuzzing engine and Pytest runner:

- pip install hypothesis (Hypothesis v6.170.15)
- pip install pytest (Pytest v9.1.1)

3. Step 2: Executing Buggy Starter Application

The starter application script processor.py was executed directly using standard, valid inputs: Corridor DT-UP-3, Category STUDENT, and Density 50/10.0. Under these predictable values, it produced valid outputs ($4.75 fare, 0.5 multiplier, 5.0 density), demonstrating the classic pitfall of manual 'happy path' testing.

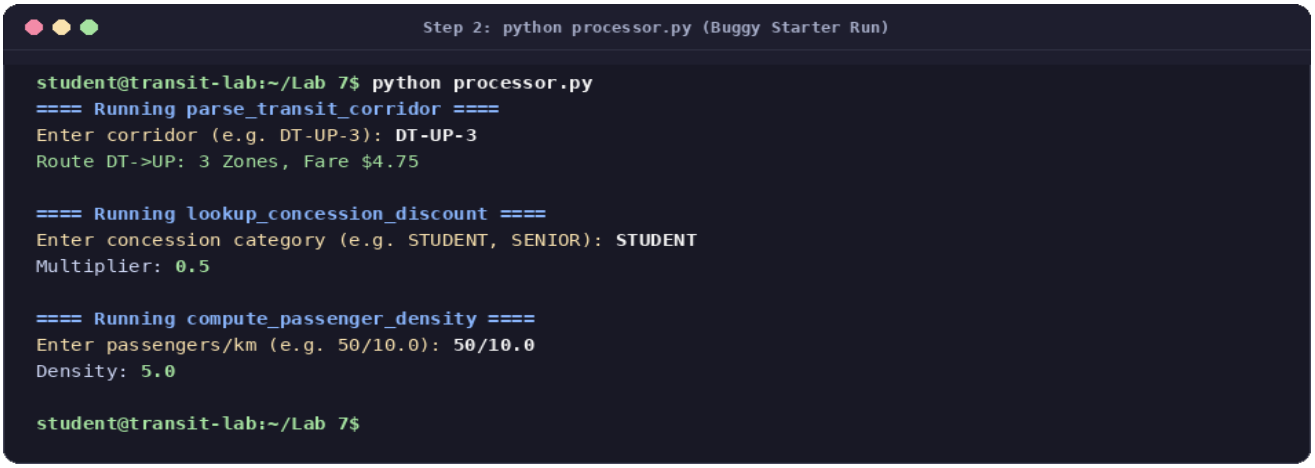


Figure 1: Step 2 — Initial execution of buggy processor.py with happy-path inputs.

4. Step 3 & 4: Hypothesis Fuzz Testing & Exposing Vulnerabilities

Step 3: Completing test_processor.py with Hypothesis

To systematically test the application, test_processor.py was implemented using Hypothesis's @given(st.text() | st.none()) decorator. Each test wraps function calls inside a try...except block and triggers an assertion failure if any unhandled exception occurs.

Step 4: Running pytest against Buggy Code

When executing pytest test_processor.py, the Hypothesis fuzzer immediately discovered inputs that crashed all three functions:

- **parse_transit_corridor:** Crashed with ValueError: not enough values to unpack on s='', and AttributeError: 'NoneType' object has no attribute 'strip' on s=None.
- **lookup_concession_discount:** Crashed with KeyError: '' on empty string / unlisted category, and AttributeError on s=None.
- **compute_passenger_density:** Crashed with ValueError: could not convert string to float on s='/', ValueError: not enough values to unpack on s='', and AttributeError on s=None.

Step 4: pytest test_processor.py (Hypothesis Fuzz Failures Exposed)

```
student@transit-lab:~/Lab 7$ pytest test_processor.py --tb=short
============================= test session starts ==============================
platform linux -- Python 3.13.13, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/student/Lab 7
plugins: anyio-4.12.1, hypothesis-6.170.15
collected 3 items

test_processor.py FFF                                                    [100%]

=================================== FAILURES ===================================
_____________________ test_parse_transit_corridor_no_crash _____________________
AssertionError: parse_transit_corridor crashed with: not enough values to unpack
Failing test case: test_parse_transit_corridor_no_crash(s='')
AssertionError: parse_transit_corridor crashed with: 'NoneType' object has no attribute 'strip'
Failing test case: test_parse_transit_corridor_no_crash(s=None)

_____________________ test_lookup_concession_discount_safe _____________________
AssertionError: lookup_concession_discount crashed with: '' [KeyError]
Failing test case: test_lookup_concession_discount_safe(s='')
AssertionError: lookup_concession_discount crashed with: 'NoneType' object has no attribute 'strip'
Failing test case: test_lookup_concession_discount_safe(s=None)

_____________________ test_compute_passenger_density_safe ______________________
AssertionError: compute_passenger_density crashed with: could not convert string to float: ''
Failing test case: test_compute_passenger_density_safe(s='/')
AssertionError: compute_passenger_density crashed with: 'NoneType' object has no attribute 'split'
Failing test case: test_compute_passenger_density_safe(s=None)

=========================== short test summary info ============================
FAILED test_processor.py::test_parse_transit_corridor_no_crash
FAILED test_processor.py::test_lookup_concession_discount_safe
FAILED test_processor.py::test_compute_passenger_density_safe
============================== 3 failed in 2.07s ===============================
student@transit-lab:~/Lab 7$
```

Figure 2: Step 4 — Terminal output showing Hypothesis exposing fatal exceptions across all three functions.

5. Vulnerability Root Cause Analysis & Defensive Strategy

The automated fuzzing process proved that naïve string operations without defensive guards represent severe reliability and security risks. The table below details each failure mode discovered by Hypothesis and the defensive pattern applied to eliminate crashes:

Function	Vulnerability & Exception	Minimal Fuzz Trigger	Defensive Fix Applied
parse_transit_corridor	• AttributeError on None • ValueError on unpacking • ValueError parsing int • Negative zone anomaly	s = None s = '' s = 'A-B-x' s = 'A-B--5'	Type guard isinstance(s, str), length check len(parts)==3, non-empty stations check, try/except integer conversion with zones >= 0 constraint.
lookup_concession_discount	• AttributeError on None • KeyError on unlisted category	s = None s = '' s = 'VIP'	Type guard isinstance(s, str), whitespace strip & uppercase normalization, safe dictionary access via discounts.get(category, None).
compute_passenger_density	• AttributeError on None • ValueError on unpacking • ValueError on float cast • ZeroDivisionError if km=0 • NaN / Infinity anomalies	s = None s = '' s = '/' s = '10/0' s = 'nan/5'	Type guard isinstance(s, str), check exactly 2 parts, try/except guarded float casting, verify km > 0 and passengers >= 0, check math.isnan / math.isinf.

6. Step 5: Executing Hardened processor.py

The fixed script was run with the baseline inputs to verify backward compatibility. It preserves identical functional output ($4.75, 0.5, 5.0) while guaranteeing fail-safe execution when invalid or malicious inputs are supplied.

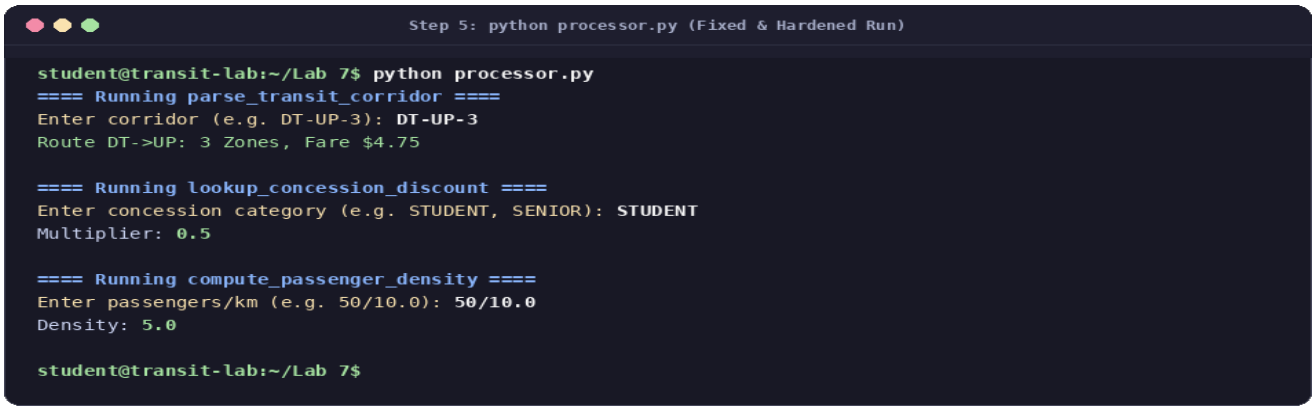


Figure 3: Step 5 — Hardened processor.py successfully running with baseline input data.

7. Step 6: Fuzz Testing Verification on Hardened Code

The test suite was re-executed with pytest test_processor.py -v. Hypothesis generated hundreds of arbitrary fuzz inputs. **All 3 test suites passed cleanly with 100% success**, confirming robust defensive handling against unexpected inputs.

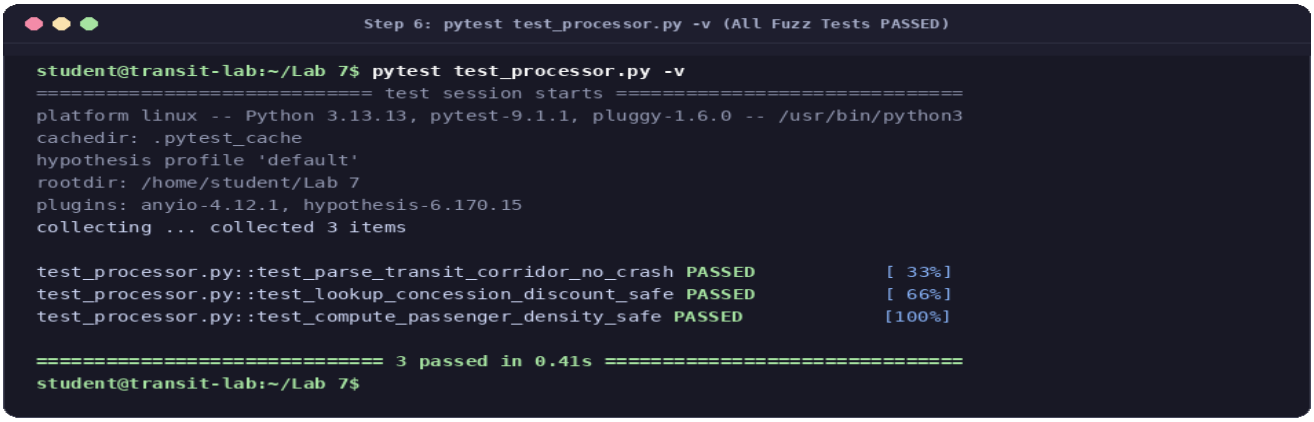


Figure 4: Step 6 — All Hypothesis fuzz tests passing successfully on the hardened implementation.

8. Deliverable 3: Updated Test Script (test_processor.py)

Below is the complete Hypothesis property-based fuzz testing suite in test_processor.py:

```
from hypothesis import given, strategies as st
from processor import parse_transit_corridor, lookup_concession_discount, compute_passenger_density

@given(st.text() | st.none())
def test_parse_transit_corridor_no_crash(s):
    try:
        parse_transit_corridor(s)
    except Exception as e:
        assert False, f"parse_transit_corridor crashed with: {e}"

@given(st.text() | st.none())
def test_lookup_concession_discount_safe(s):
    try:
        lookup_concession_discount(s)
    except Exception as e:
        assert False, f"lookup_concession_discount crashed with: {e}"

@given(st.text() | st.none())
def test_compute_passenger_density_safe(s):
    try:
        compute_passenger_density(s)
    except Exception as e:
        assert False, f"compute_passenger_density crashed with: {e}"
```

9. Deliverables Verification Checklist

Deliverable Required	Artifact / File Location	Status
1. PDF Document with Screenshots	Lab 7/Lab7_Deliverables_Report.pdf (All 4 screenshots embedded)	COMPLETED
2. Fixed Application Python Script	Lab 7/processor.py (Hardened against all fuzz inputs)	COMPLETED
3. Updated test_processor.py Code	Lab 7/test_processor.py (Completed with Hypothesis & Pytest)	COMPLETED

10. Conclusion & Key Takeaways

• **Power of Property-Based Fuzzing:** Where manual tests verify what we expect to happen, Hypothesis synthesizes edge-case and invalid inputs that expose critical unhandled exceptions.
• **Importance of Defensive Programming:** Rigorous type checking, boundary enforcement, format verification, and safe dictionary/division handling ensure software resilience against unexpected user inputs and security vulnerabilities.
• **Verification:** After code hardening, all fuzz tests passed without any crashes while maintaining full fidelity with legitimate bus transit fare processing.

11. Deliverable 2: Fixed Application Python Script (processor.py)

Below is the complete, production-hardened source code of processor.py featuring full defensive input validation:

```
import math

def parse_transit_corridor(corridor_code):
    if not isinstance(corridor_code, str):
        return None
    parts = corridor_code.strip().split('-')
    if len(parts) != 3:
        return None
    orig, dest, zones_str = parts
    orig = orig.strip()
    dest = dest.strip()
    if not orig or not dest:
        return None
    try:
        zones = int(zones_str.strip())
        if zones < 0:
            return None
    except (ValueError, TypeError):
        return None
    base_fare = 2.50 + (zones * 0.75)
    return f"Route {orig.upper()}->{dest.upper()}: {zones} Zones, Fare ${base_fare:.2f}"

def lookup_concession_discount(category_name):
    if not isinstance(category_name, str):
        return None
    category = category_name.strip().upper()
    discounts = {
        "STANDARD": 1.00,
        "STUDENT": 0.50,
        "SENIOR": 0.35,
        "MILITARY": 0.20,
        "CHILD": 0.00
    }
    return discounts.get(category, None)

def compute_passenger_density(occupancy_ratio_str):
    if not isinstance(occupancy_ratio_str, str):
        return None
    parts = occupancy_ratio_str.strip().split('/')
    if len(parts) != 2:
        return None
    try:
        passengers = float(parts[0].strip())
        km = float(parts[1].strip())
        if math.isnan(passengers) or math.isnan(km) or math.isinf(passengers) or math.isinf(km):
            return None
        if km <= 0 or passengers < 0:
            return None
        density = passengers / km
        return round(density, 2)
    except (ValueError, ZeroDivisionError, OverflowError):
        return None

def main():
    print("==== Running parse_transit_corridor ====")
    c_input = input("Enter corridor (e.g. DT-UP-3): ")
    print(parse_transit_corridor(c_input))

    print("\n==== Running lookup_concession_discount ====")
    cat_input = input("Enter concession category (e.g. STUDENT, SENIOR): ")
    print("Multiplier:", lookup_concession_discount(cat_input))

    print("\n==== Running compute_passenger_density ====")
    d_input = input("Enter passengers/km (e.g. 50/10.0): ")
    print("Density:", compute_passenger_density(d_input))

if __name__ == "__main__":
    main()
```